SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Comprehensive System Design and Implementation Challenge

Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Comprehensive System Design and Implementation Challenge – District Blood Bank Network
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	13 NOVEMBER 2026

TASK A4 — Design Dynamic Memory Use (4 Marks)

Task / Problem Statement

The number of donors is unknown (50 to 50,000 or more). Store them in memory without wasting space and without losing data.

Aim

To design a growable array that doubles with realloc, handle allocation failure, and avoid memory errors.

A4.1 – Growable Donor List (2 marks)

```
typedef struct { Donor *data; int size, capacity; } DonorList;   /* starts as {NULL, 0, 0} */
```

```
ALGORITHM AddDonor(list*, donor)
  INPUT  : list (data, size, capacity), donor record
  OUTPUT : 1 on success, 0 if memory could not be obtained (list unchanged)
BEGIN
  IF list.size = list.capacity THEN                // list is full
    IF list.capacity = 0 THEN newCap <- 2 ELSE newCap <- 2 * list.capacity
    p <- realloc(list.data, newCap * sizeof(Donor))   // temporary pointer
    IF p = NULL THEN
      RETURN 0                                     // old block still valid, no data lost
    END IF
    list.data <- p ; list.capacity <- newCap
  END IF
  list.data[list.size] <- donor
  list.size <- list.size + 1
  RETURN 1
END
```

If realloc() fails it returns NULL and leaves the old block untouched. Because the result is first stored in the temporary p, list->data still points to the old, valid block: the donors already in memory stay usable, the function reports failure (return 0), the caller prints an error and can still save the file and free the list. Writing list->data = realloc(list->data, ...) directly would overwrite the only pointer with NULL, leak the old block and lose all donors.

A4.2 – Memory Errors (2 marks)

Error	Meaning	How it could happen in the blood bank	How the design prevents it
Memory leak	Allocated memory is never freed, so it is lost until the program ends	find loads donors into a list and returns early on an error without calling free; or list->data = realloc(list->data, n) fails and the old block is lost	Single exit path: main always calls free_list(&list); temporary pointer p in add_donor; checked with valgrind (“All heap blocks were freed”)
Dangling pointer	A pointer that still holds the address of memory that was freed or moved	Donor *d = &list.data[0]; then more donors are added; realloc moves the block; d->id now reads freed memory. Also free(list.data) followed by reading list.data[0]	Never keep Donor * across add_donor; use indices. free_list sets data = NULL, size = capacity = 0. lowest_stock points into a fixed array that is never reallocated
Double free	free called twice on the same block → heap corruption / crash	An error path calls free_list(&list) and then main calls it again at the end	free_list sets data = NULL afterwards, and free(NULL) is harmless; one owner (main) for the list

B2 reproduces all three on purpose and shows valgrind’s report for each.

Alternative Concept and Complexity Analysis – grow by a fixed 10 records

The solution would be newCap = capacity + 10. Every 10th insertion triggers a realloc, and (in the worst case, when the block must move) all existing elements are copied.

Donors n	Elements copied: doubling	Elements copied: +10 each time	Unused slots at the end: doubling	Unused: +10
1,000	1,022	49,500	24	≤ 9
10,000	16,382	4,995,000	6,384	≤ 9
100,000	131,070	499,950,000	31,072	≤ 9
1,000,000	1,048,574	49,999,500,000	48,576	≤ 9

(Counts computed by simulating both strategies; they are worst-case counts, because a real realloc can sometimes extend the block in place.)

- **Doubling:** copies $1 + 2 + 4 + \ldots < 2n$ elements in total → **O(n)** overall, O(1) amortised per insertion. Cost: up to almost half of the capacity can be unused (capacity < 2n, so waste < n elements).
- **Fixed +10:** copies $10 + 20 + 30 + \ldots \approx n^2/20$ elements → **$O(n^2)$**. At 100,000 donors that is 500 million element copies against 131 thousand. Waste is only 9 elements at most.

Conclusion: doubling trades a little memory for a huge saving in time, so it is used.

Result

A growable list with safe realloc handling was designed, three memory errors were explained with prevention, and the growth strategies were compared (O(n) against $O(n^2)$).